

PART

IV

부산시 인구 격자별 분석 (Python)





IV | 부산시 인구 격자별 분석

SGIS 자료제공 메뉴에서 내려받은 부산시 경계(`bnd_sido_21_2024_2Q.shp`), 격자 경계(`grid_마라_1K.shp` , `grid_마마_1K.shp`), 2023년 격자 통계(`2023년_인구_마라_1K.csv` , `2023년_인구_마마_1K.csv`)를 파이썬(Jupyter Notebook)으로 분석하는 과정을 단계별로 설명합니다.

1 분석 라이브러리 로드 및 사용 준비하기

- ▶ geopandas, pandas, matplotlib 라이브러리를 임포트합니다. 각각 공간 데이터 분석, 표 형식 데이터 처리, 시각화에 사용됩니다.

```
[1]: import geopandas as gpd # 지리정보 파일 읽고 처리
import pandas as pd # 표 형식 데이터 처리
import matplotlib.pyplot as plt # 그래프, 지도 등 시각화
```

▶ `plt.rcParams['font.family'] = 'Malgun Gothic'` # 윈도우에서 한글 폰트 설정

그래프, 지도 등 시각화에 사용되는 matplotlib는 사용하기 전 한글 폰트를 명시적으로 설정해야 합니다. 한글이 사용되는 경우, 폰트 파일 로딩 지연 문제 등으로 소스코드 첫부분에서 실행해줍니다.

2 부산시 경계, 격자 경계 및 통계 데이터 불러오기

- ▶ `gpd.read_file()`로 경계 데이터를 불러오고, `pd.read_csv()`로 인구 통계 데이터를 불러옵니다.
`bord_sido( bnd_sido_21_2024_2Q.shp ), grid_mara( grid_마라_1K.shp ), grid_mama( grid_마마_1K.shp ), stat_mara( 2023년_인구_마라_1K.csv ), stat_mama( 2023년_인구_마마_1K.csv )`로 각각 저장합니다. (※ 경로는 사용자 PC에 맞게 수정)

```
[2]: prj_dir = 'C:/SGIS/Python/부산시 인구 격자별 분석'
bord_sido = gpd.read_file(prj_dir + '/' + 'bnd_sido_21_2024_2Q.shp')
grid_mara = gpd.read_file(prj_dir + '/' + 'grid_마라_1K.shp')
grid_mama = gpd.read_file(prj_dir + '/' + 'grid_마마_1K.shp')
stat_mara = pd.read_csv(prj_dir + '/' + '2023년_인구_마라_1K.csv', encoding='CP949',
                        names=['BASE_YEAR', 'GRID_1K_CD', 'STAT_CD', 'POP'])
stat_mama = pd.read_csv(prj_dir + '/' + '2023년_인구_마마_1K.csv', encoding='CP949',
                        names=['BASE_YEAR', 'GRID_1K_CD', 'STAT_CD', 'POP'])
```

※ 통계 파일을 열어보면 구분자는 '(콤마)'이고, 첫 행에 컬럼명이 없는 것을 확인할 수 있습니다. 이런 경우 파일을 불러오면서 컬럼명을 함께 지정할 수 있습니다.

3 격자 경계 및 통계 데이터 병합하기

- ▶ **grid_마라** 경계와 통계 및 **grid_마마** 경계와 통계 데이터를 각각 병합합니다.

```
[3]: grid_merged = pd.concat([grid_mara, grid_mama], ignore_index=True)
      stat_merged = pd.concat([stat_mara, stat_mama], ignore_index=True)
```

4 데이터 구조 확인하기

- ▶ 경계와 통계 각각의 파일이 잘 병합되었는지, 데이터 타입은 무엇인지 등 구조를 확인합니다.

```
[4]: print(grid_merged.info()) # 격자 경계 코드(GRID_1K_CD), 지리정보(geometry)로 구성
      print(stat_merged.info()) # 기준년도(BASE_YEAR), 격자 경계 코드(GRID_1K_CD)
                                   # 통계 코드(STAT_CD), 인구수(POP)
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 9549 entries, 0 to 9548
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   GRID_1K_CD  9549 non-null   object
1   geometry    9549 non-null   geometry
dtypes: geometry(1), object(1)
memory usage: 149.3+ KB
None
```

➤ 데이터프레임은 총 9,549개의 행으로 구성되며, 2개의 열이 있습니다. 각각의 열에는 비어 있는 값이 없으며, 데이터 타입은 object(문자열)입니다.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 261928 entries, 0 to 261927
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   BASE_YEAR    261928 non-null  int64
1   GRID_1K_CD   261928 non-null  object
2   STAT_CD      261928 non-null  object
3   POP          261928 non-null  int64
dtypes: int64(2), object(2)
memory usage: 8.0+ MB
None
```

➤ 데이터프레임은 총 261,928개의 행으로 구성되며, 4개의 열이 있습니다. 각각의 열에는 비어 있는 값이 없으며, 데이터 타입은 int64(정수형), object(문자열)입니다.

※ GRID_1K_CD의 데이터 타입(object)이 동일하므로 추후 데이터 타입 일치 과정 없이 바로 결합이 가능합니다.

※ 추가로 print(grid_mara.info())와 print(grid_mama.info()), print(stat_mara.info())와 print(stat_mama.info())를 통해 병합 전의 각각 데이터 수, 데이터 타입 등을 확인할 수 있습니다.

5 통계 항목 필터링

- ▶ STAT_CD 컬럼에 있는 고유값을 확인하고 분석에 필요한 총인구('to_in_001')만 필터링하여 stat_total에 저장합니다.

```
[5]: # STAT_CD 열에 있는 고유 값 확인
unique_stat_cd = stat_merged['STAT_CD'].unique()
print(unique_stat_cd)
```

```
['in_age_005' 'in_age_006' 'in_age_007' 'in_age_009' 'in_age_010'
'in_age_011' 'in_age_012' 'in_age_013' 'in_age_014' 'in_age_015'
'in_age_016' 'in_age_017' 'in_age_035' 'in_age_036' 'in_age_039'
'in_age_041' 'in_age_042' 'in_age_043' 'in_age_044' 'in_age_045'
'in_age_046' 'in_age_047' 'in_age_066' 'in_age_067' 'in_age_070'
'in_age_071' 'in_age_072' 'in_age_073' 'in_age_074' 'in_age_075'
'in_age_076' 'in_age_077' 'to_in_001' 'to_in_007' 'to_in_008'
'in_age_001' 'in_age_008' 'in_age_031' 'in_age_037' 'in_age_038'
'in_age_061' 'in_age_003' 'in_age_004' 'in_age_018' 'in_age_019'
'in_age_020' 'in_age_040' 'in_age_048' 'in_age_063' 'in_age_064'
'in_age_069' 'in_age_078' 'in_age_079' 'in_age_080' 'in_age_033'
'in_age_034' 'in_age_068' 'in_age_002' 'in_age_062' 'in_age_065'
'in_age_032' 'in_age_049' 'in_age_021' 'in_age_050' 'in_age_081'
'in_age_051']
```

```
[6]: # 총인구('to_in_001') 데이터만 필터링
stat_total = stat_merged[stat_merged['STAT_CD'] == 'to_in_001']
# 필터링 후 STAT_CD 고유값 재확인
unique_stat_cd = stat_total['STAT_CD'].unique()
print(unique_stat_cd)
```

```
['to_in_001']
```

6 분석에 필요한 컬럼만 선택하기

- ▶ stat_total에서 GRID_1K_CD와 POP 컬럼만 남겨서 나머지 불필요한 열을 제거합니다.(추후 경계 파일과 병합 과정에서 필요한 컬럼만 저장)

```
[7]: stat_total = stat_total[['GRID_1K_CD', 'POP']]
```

7 부산시 경계와 격자 경계가 교차하는 격자만 선택하기

```
[8]: grid_intersects = grid_merged[grid_merged.geometry.intersects(bord_sido.geometry.union_all())]
```

- ▶ grid_mara와 grid_mama는 부산과 경상도 일대를 포함하는 1km 격자 경계이며, 부산시 인구 분석을 위해서 부산시 경계와 교차하는 격자 경계만 추출합니다.

8 경계와 통계 병합하기

- ▶ grid_intersects와 stat_total 두 개의 데이터프레임을 'GRID_1K_CD'를 기준으로 left join(왼쪽 조인)하여 병합합니다.

```
[9]: grid_intersects = grid_intersects.merge(stat_total, on='GRID_1K_CD', how='left')
# 결합 후 통계값이 없는 구간에 NaN이 들어가 있을 수 있음(결측값 확인)
print(grid_intersects.isna().sum())
```

```
GRID_1K_CD    0
geometry      0
POP          244
dtype: int64
```

```
[10]: # 결측값을 0으로 채우기
grid_intersects['POP'] = grid_intersects['POP'].fillna(0)
```

※ 「SGIS 자료제공」의 격자 통계 특성 상, 인구 또는 사업체가 없는 격자에 대해서는 통계 데이터를 생성하지 않으므로 결측값을 '0'으로 채울 수 있습니다.

9 총인구수를 기준으로 구간 나누기

- ▶ 총인구수('POP')를 5개의 구간으로 나누고 각 구간에 대한 라벨을 지정합니다.

```
[11]: # 인구 구간 직접 지정하여 나누기
print(grid_intersects['POP'].min(), grid_intersects['POP'].max()) # 최소값과 최대값 확인
```

```
0.0 31611.0
```

```
[12]: bins = [0, 1000, 5000, 10000, 20000, 32000] # 적절한 구간 지정
labels = ['0~1000', '1000~5000', '5000~10,000', '10,000~20,000', '20,000~32,000'] # 구간의 라벨
# 각 인구 수에 대해 구간 할당
grid_intersects['POP_BINS'] = pd.cut(grid_intersects['POP'], bins=bins, labels=labels, right=False)
print(grid_intersects.head()) # 인구 구간('POP_BINS') 컬럼 추가 확인
```

	GRID_1K_CD	geometry	POP	POP_BINS
0	마라1569	POLYGON ((1115000 1669000, 1115000 1670000, 11...	0.0	0~1000
1	마라1668	POLYGON ((1116000 1668000, 1116000 1669000, 11...	0.0	0~1000
2	마라1669	POLYGON ((1116000 1669000, 1116000 1670000, 11...	0.0	0~1000
3	마라1773	POLYGON ((1117000 1673000, 1117000 1674000, 11...	0.0	0~1000
4	마라1774	POLYGON ((1117000 1674000, 1117000 1675000, 11...	0.0	0~1000

10 지도 시각화 하기

```
[13]: # 시각화 설정(그림과 축 설정)
fig, ax = plt.subplots(1, 1, figsize=(12, 12))
# 격자 시각화(격자 경계 선은 연하게 설정)
grid_intersects.plot(column='POP_BINS', ax=ax, legend=True,
                      cmap='OrRd', edgecolor='black', linewidth=0.5, alpha=0.7)
# 부산시 경계 추가 (경계선만 보이도록 설정, 배경은 투명)
bord_sido.plot(ax=ax, color='none', edgecolor='black', linewidth=2)

# (추가) 부산시 시군구 경계를 추가하여 명확하게 분석하기
bord_sgg = gpd.read_file(prj_dir + '/' + 'bnd_sigungu_21_2024_2Q.shp')
bord_sgg.plot(ax=ax, color='none', edgecolor='black', linewidth=2)
bord_sgg['centroid'] = bord_sgg.geometry.centroid # 중심점 계산
for idx, row in bord_sgg.iterrows():
    ax.text(row['centroid'].x, row['centroid'].y, row['SIGUNGU_NM'], fontsize=15, ha='center',
            color='black', bbox=dict(facecolor='white', alpha=0.7, edgecolor='none',
                                    boxstyle='round,pad=0.1'))

# 범례 위치 및 제목 설정
ax.get_legend().set_bbox_to_anchor((0.5, -0.05)) # 범례 위치 조정
ax.get_legend().set_title("<인구 범위>") # 범례 제목 설정
ax.set_title("<부산시 인구 격차별 분석>", fontsize=16) # 지도 제목 설정

# 레이아웃 조정 및 출력
plt.tight_layout() # 그래프 레이아웃이 겹치지 않도록 자동 조정
plt.savefig(prj_dir + '/' + '부산시 인구 격차별 분석 지도.png')
plt.show() # 시각화 결과 화면에 출력
```

| 부산시 인구 격차별 분석 |

